The idea of CSP is simple: if there's no shared data, each thread can be reasoned about entirely independently, purely on the basis of how it behaves in response to the messages that it received. Each thread is therefore effectively a state machine: when it receives a message, it updates its state in some manner and maybe sends one or more messages to other threads, with the processing performed depending on the initial state. One way to write such threads would be to formalize this and implement a Finite State Machine model, but this isn't the only way; the state machine can be implicit in the structure of the application. Which method works better in any given scenario depends on the exact behavioral requirements of the situation and the expertise of the programming team. However you choose to implement each thread, the separation into independent processes has the potential to remove much of the complication from shared-data concurrency and therefore make programming easier, lowering the bug rate.

True communicating sequential processes have no shared data, with all communication passed through the message queues, but because C++ threads share an address space, it's not possible to enforce this requirement. This is where the discipline comes in: as application or library authors, it's our responsibility to ensure that we don't share data between the threads. Of course, the message queues must be shared in order for the threads to communicate, but the details can be wrapped in the library.

Imagine for a moment that you're implementing the code for an ATM. This code needs to handle interaction with the person trying to withdraw money and interaction with the relevant bank, as well as control the physical machinery to accept the person's card, display appropriate messages, handle key presses, issue money, and return their card.

One way to handle everything would be to split the code into three independent threads: one to handle the physical machinery, one to handle the ATM logic, and one to communicate with the bank. These threads could communicate purely by passing messages rather than sharing any data. For example, the thread handling the machinery would send a message to the logic thread when the person at the machine entered their card or pressed a button, and the logic thread would send a message to the machinery thread indicating how much money to dispense, and so forth.

One way to model the ATM logic would be as a state machine. In each state the thread waits for an acceptable message, which it then processes. This may result in transitioning to a new state, and the cycle continues. The states involved in a simple implementation are shown in figure 4.3. In this simplified implementation, the system waits for a card to be inserted. Once the card is inserted, it then waits for the user to enter their PIN, one digit at a time. They can delete the last digit entered. Once enough digits have been entered, the PIN is verified. If the PIN is not OK, you're finished, so you return the card to the customer and resume waiting for someone to enter their card. If the PIN is OK, you wait for them to either cancel the transaction or select an amount to withdraw. If they cancel, you're finished, and you return their card. If they select an amount, you wait for confirmation from the bank before issuing the cash and returning the card or displaying an "insufficient funds" message and returning their card. Obviously, a real ATM is considerably more complex, but this is enough to illustrate the idea.

Figure 4.3. A simple state machine model for an ATM